



Chapter 7: Normalization

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use

What is Normalization

- process of organizing data in a database to minimize redundancy and dependency

Advantages

- improving data integrity
- reducing data redundancy
- making the database more efficient in terms of storage and retrieval

over-normalization can also lead to performance issues

Anomalies

unexpected or undesirable outcomes that can occur when data is not properly organized or structured

- Insert anomaly
- Delete anomaly
- Update anomaly

Features of Good Relational Designs

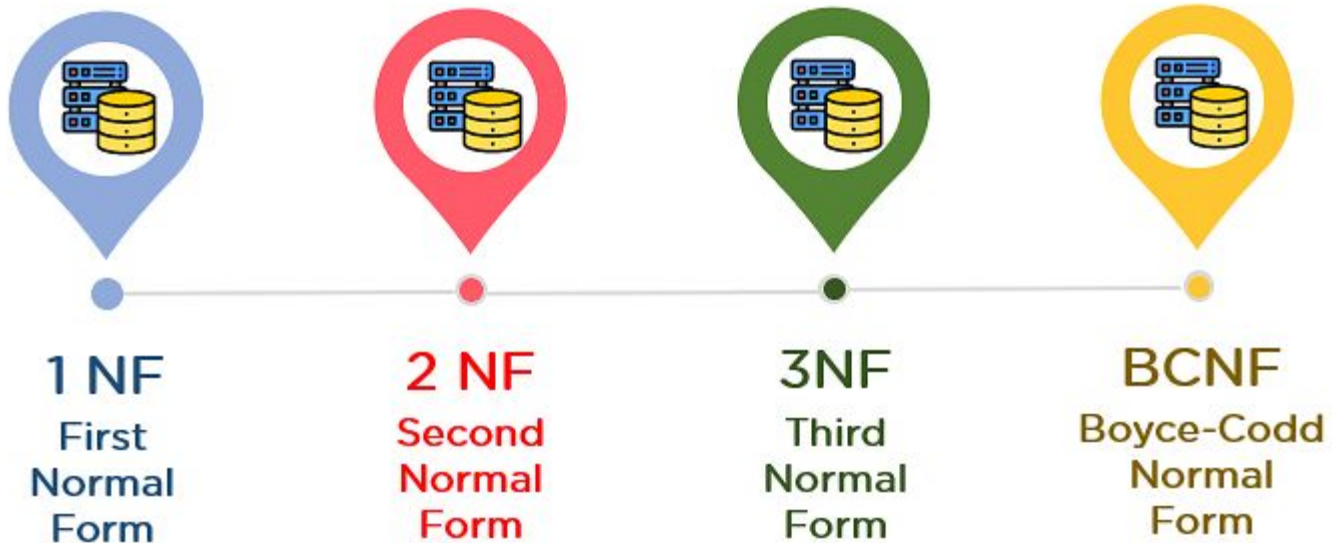
- Suppose we combine *instructor* and *department* into *in_dep*, which represents the natural join on the relations *instructor* and *department*

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- There is repetition of information
- Need to use null values (if we add a new department with no instructors)



Normalization





1 nf

- **Atomic Values:** Each cell (attribute) must contain **only one value**—no sets, arrays, or lists.
- **Unique Rows:** Each row must be **unique**—no duplicate rows.
- **Same Data Type in Columns:** All values in a column should be of the **same data type**.
- **Order Doesn't Matter:** The order of rows and columns does not affect the table's design.



Normalization-2NF

- 2NF Requirements:
 - Relation must be in 1NF.
 - No Partial Dependency should exist. That means, all non prime attributes should be fully functionally dependent on the candidate key not on part of the candidate key.
- Example:
- R(ABCDEF)
- FD:
 - C- $\rightarrow$ F ,
 - E- $\rightarrow$ A ,
 - EC- $\rightarrow$ D,
 - E- $\rightarrow$ B

Assume, Candidate key: EC

Prime attributes: E,C

Non- Prime attributes: A,B,D,F

Here, E- $\rightarrow$ A , C- $\rightarrow$ F , E- $\rightarrow$ B possess partial dependency as R.H.S is non prime attr and L.H.S is a part of candidate key.

So, this relation is not in 2NF.



Normalization-3NF

- 3NF Requirements:
 - Relation must be in 2NF. (No partial dependency)
 - There should be no transitive dependency. That means no non prime attribute should determine other non prime attributes.
- Example:
- R(ABCD)
- FD:
AB- $\rightarrow$ C
C- $\rightarrow$ D

Candidate key: AB

Prime attributes: A,B

Non- Prime attributes: C,D

Here, C- $\rightarrow$ D : C and D both non prime attr.

Tricks: table will be in 3NF , if for each FD

- L.H.S is a candidate key.

Or

- R.H.S is a Prime Attribute.
- Is R in 3NF?



Normalization-BCNF (0% redundancy)

- BCNF (special case of 3NF but more restricted) Requirements:
 - Relation should be in 3NF
 - X should be a superkey/candidate key for every functional dependency (FD)
 $X \rightarrow Y$ in a given relation.

- Example:
- $R(ABCD)$
- FD:
 $AB \rightarrow C$
 $D \rightarrow A$

Candidate key: AB, BD

Prime attributes: A, B, D

Non- Prime attributes: C

Tricks: table will be in BCNF , if for each FD

- L.H.S is a candidate/super key.

So, is R in BCNF??

A Normalisation Example

Take the following table.
StudentID is the primary key.

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
					Maths	\$50	A
					Info Tech	\$100	B+

Is it 1NF?

**No. There are repeating groups
(subject, subjectcost, grade)**

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
					Maths	\$50	A
					Info Tech	\$100	B+

How can you make it 1NF?

Create new rows so each cell contains
only one value

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
					Maths	\$50	A
					Info Tech	\$100	B+



StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

But now look – is the *studentID* primary
key still valid?

No – the studentID no longer uniquely identifies each row

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

You now need to declare *studentID* **and** *subject* **together** to uniquely identify each row.

So the new **key** is StudentID *and* Subject.

So. We now have 1NF.

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

Is it 2NF?

**Studentname and address are dependent
on studentID (which is part of the key)
This is good.**

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

But they are **not** dependent on
Subject (the *other* part of the
key)

And 2NF requires...

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

All non-key fields are
dependent on the ENTIRE
key (studentID + subject)

So it's not 2NF

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

How can we fix it?

Make new tables

- Make a new table for each primary key field
- Give each new table its own primary key
- Move columns from the original table to the new table that matches their primary key...

Step 1

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

STUDENT TABLE (key = StudentID)

Step 2

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Step 3

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

RESULTS TABLE (key = StudentID+Subject)

Step 3

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
19594332X	Mary Watson	10 Charles Street	Bob	Red	Maths	\$50	A
19594332X	Mary Watson	10 Charles Street	Bob	Red	Info Tech	\$100	B+

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

RESULTS TABLE (key = StudentID+Subject)

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

Step 4 - relationships

STUDENT TABLE (key = StudentID)

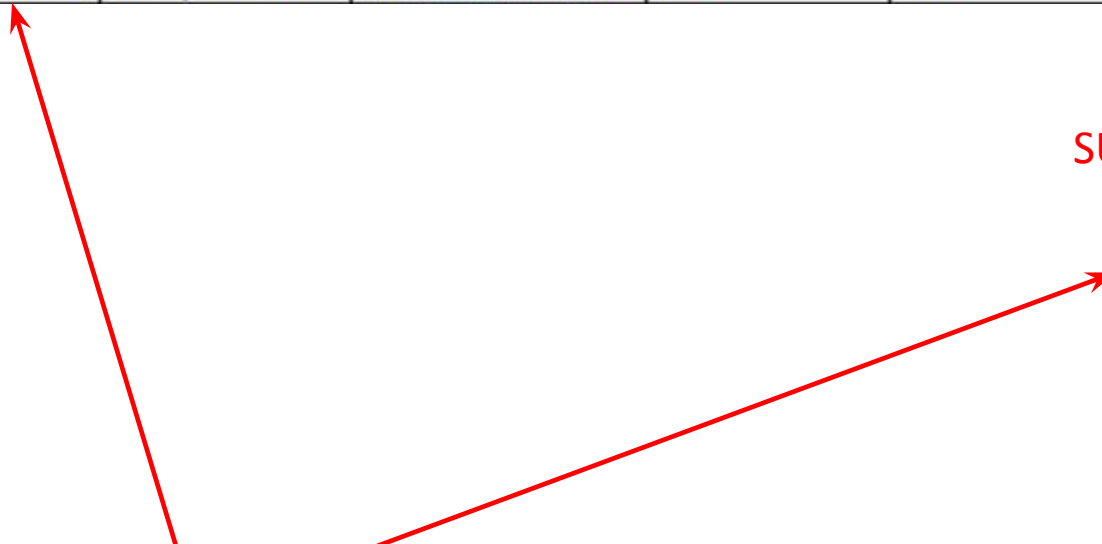
StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

1

Each student can only appear
ONCE in the student table

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

Each subject can only appear
ONCE in the subjects table

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

A subject can be listed MANY times in the results table (for different students)

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

1

1

∞

Step 4 - cardinality

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

A student can be listed MANY times in the results table (for different subjects)

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

1

1

∞

∞

A 2NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

SubjectCost is only dependent on the primary key, *Subject*



A 2NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

Grade is only dependent
on the primary key
(*studentID* + *subject*)

RESULTS TABLE (key = StudentID+Subject)



A 2NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

Name, Address are only dependent on the primary key (*StudentID*)

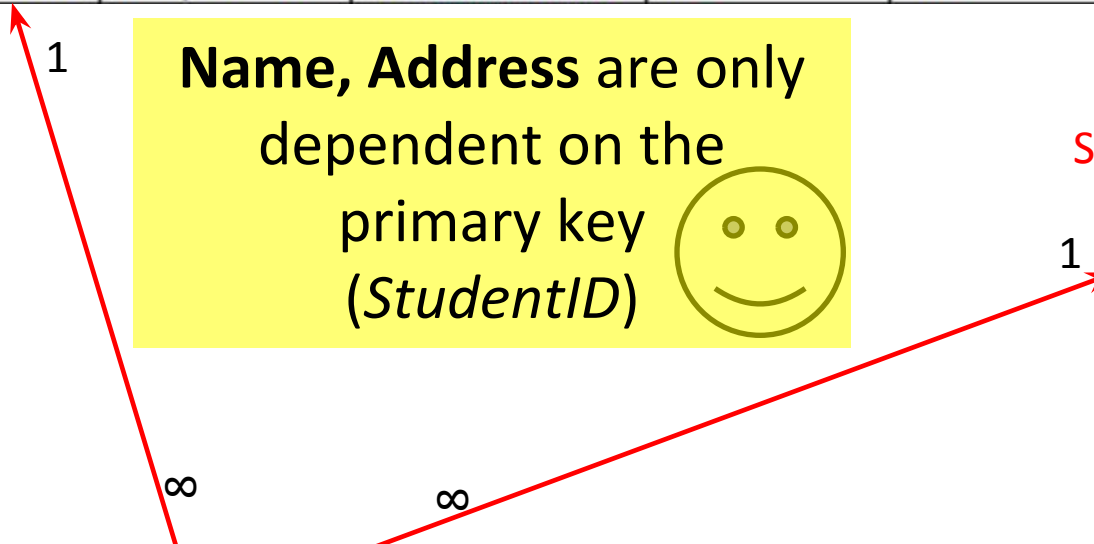


SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

**So it is
2NF!**

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

**But is it
3NF?**

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

1

∞

∞

1

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

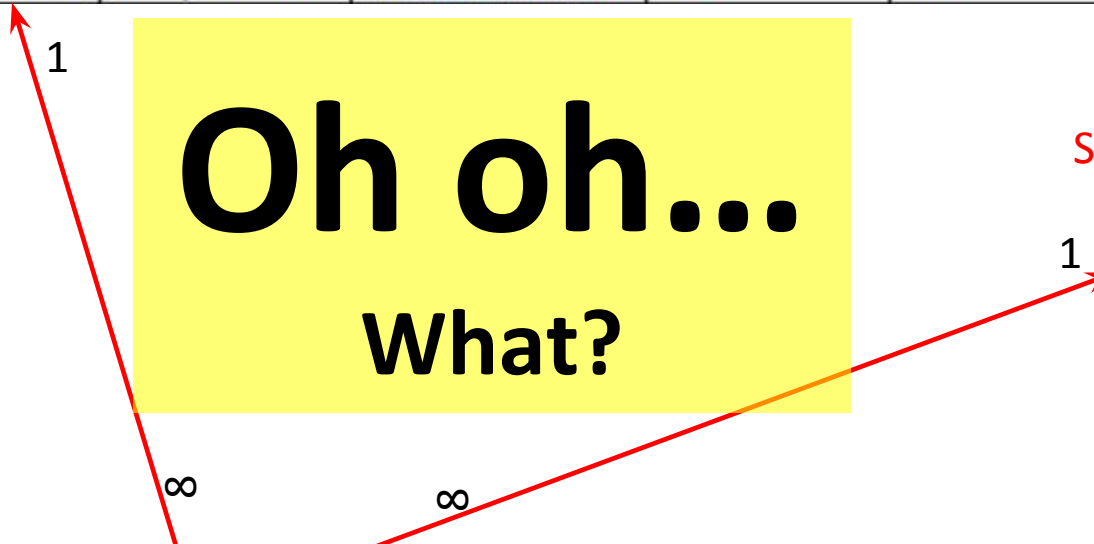
Oh oh...
What?

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

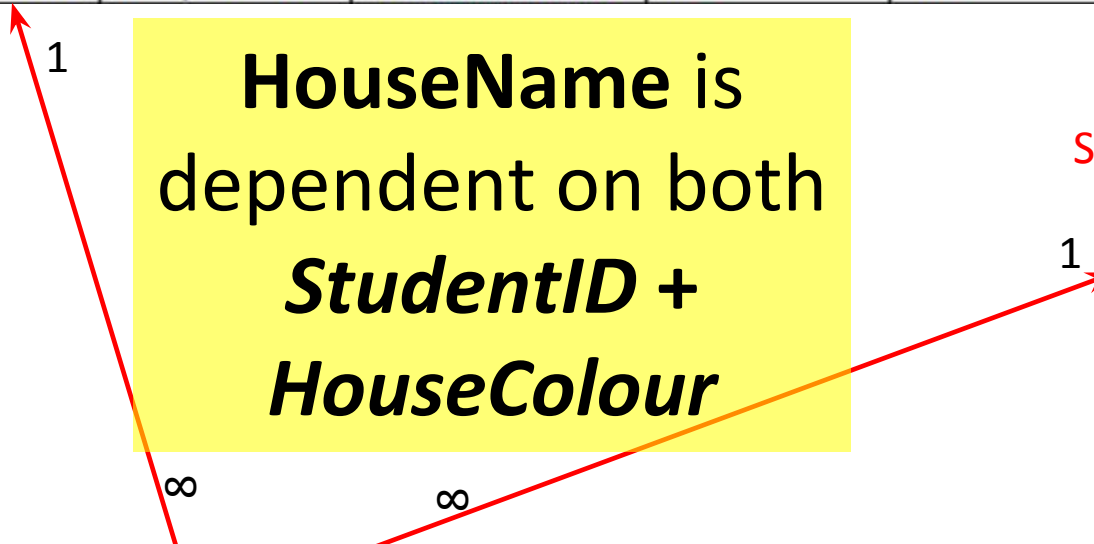
HouseName is
dependent on both
***StudentID +
HouseColour***

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

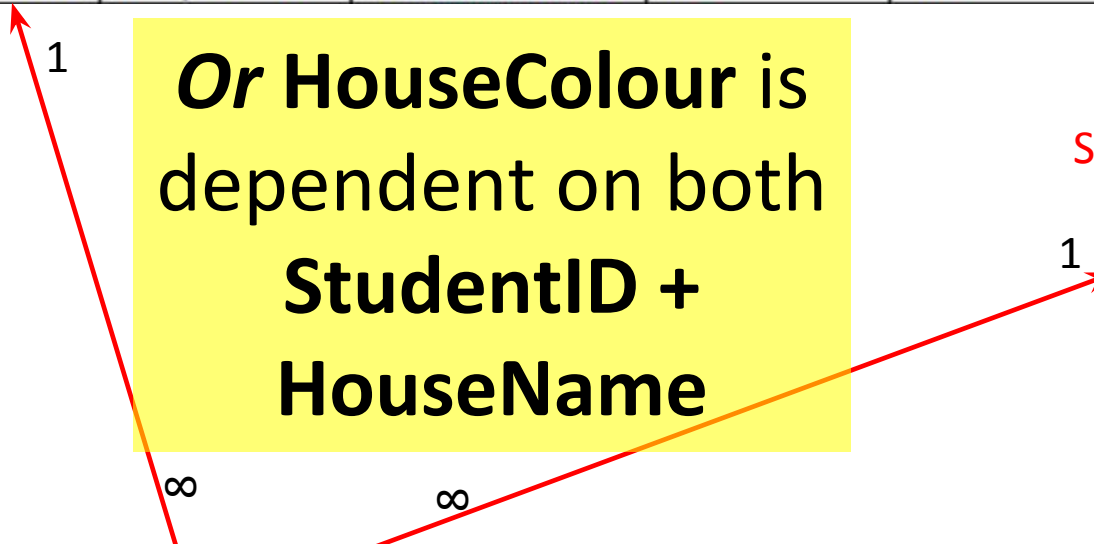
**Or HouseColour is
dependent on both
StudentID +
HouseName**

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

*But either way,
non-key fields are
dependent on MORE
THAN THE PRIMARY
KEY (studentID)*

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)

A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

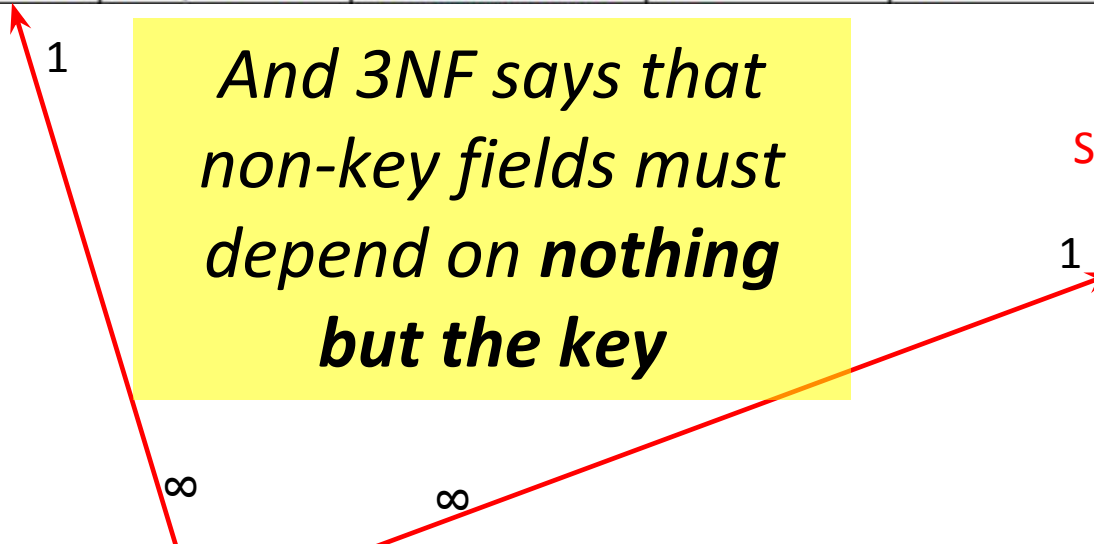
*And 3NF says that
non-key fields must
depend on **nothing**
but the key*

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



A 3NF check

STUDENT TABLE (key = StudentID)

StudentID	StudentName	Address	HouseName	HouseColor
19594332X	Mary Watson	10 Charles Street	Bob	Red

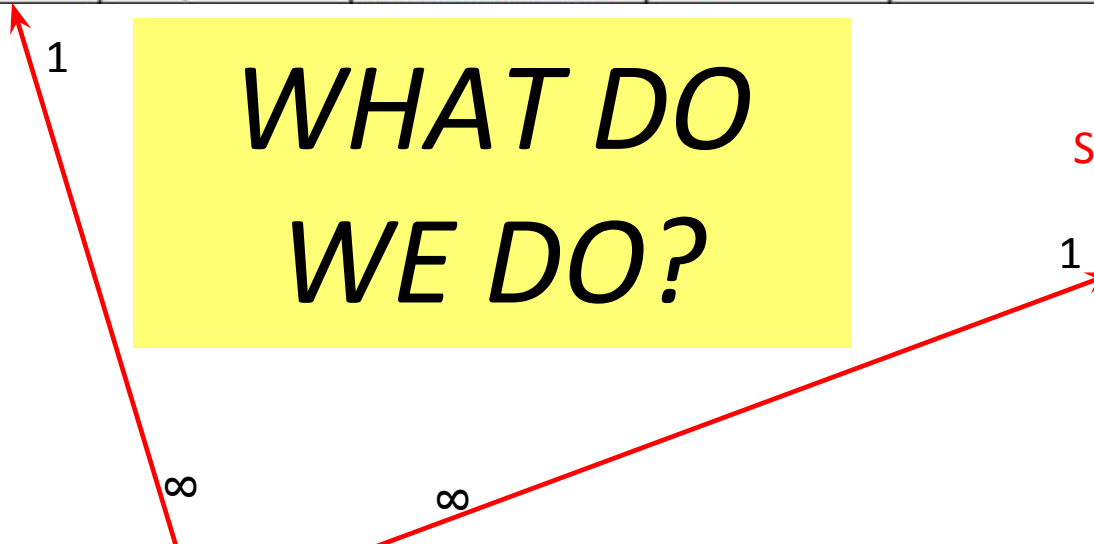
*WHAT DO
WE DO?*

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key = StudentID+Subject)



Again, carve off the offending fields

StudentTable

StudentID	StudentName	Address	HouseName
19594332X	Mary Watson	10 Charles Street	Bob

Primary key: StudentID

1

∞

∞

1

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

RESULTS TABLE (key = StudentID+Subject)

A 3NF fix

StudentTable

StudentID	StudentName	Address	HouseName
19594332X	Mary Watson	10 Charles Street	Bob

Primary key: StudentID

HouseTable

HouseName	HouseColor
Bob	Red

Primary key: HouseName

1

∞

∞

1

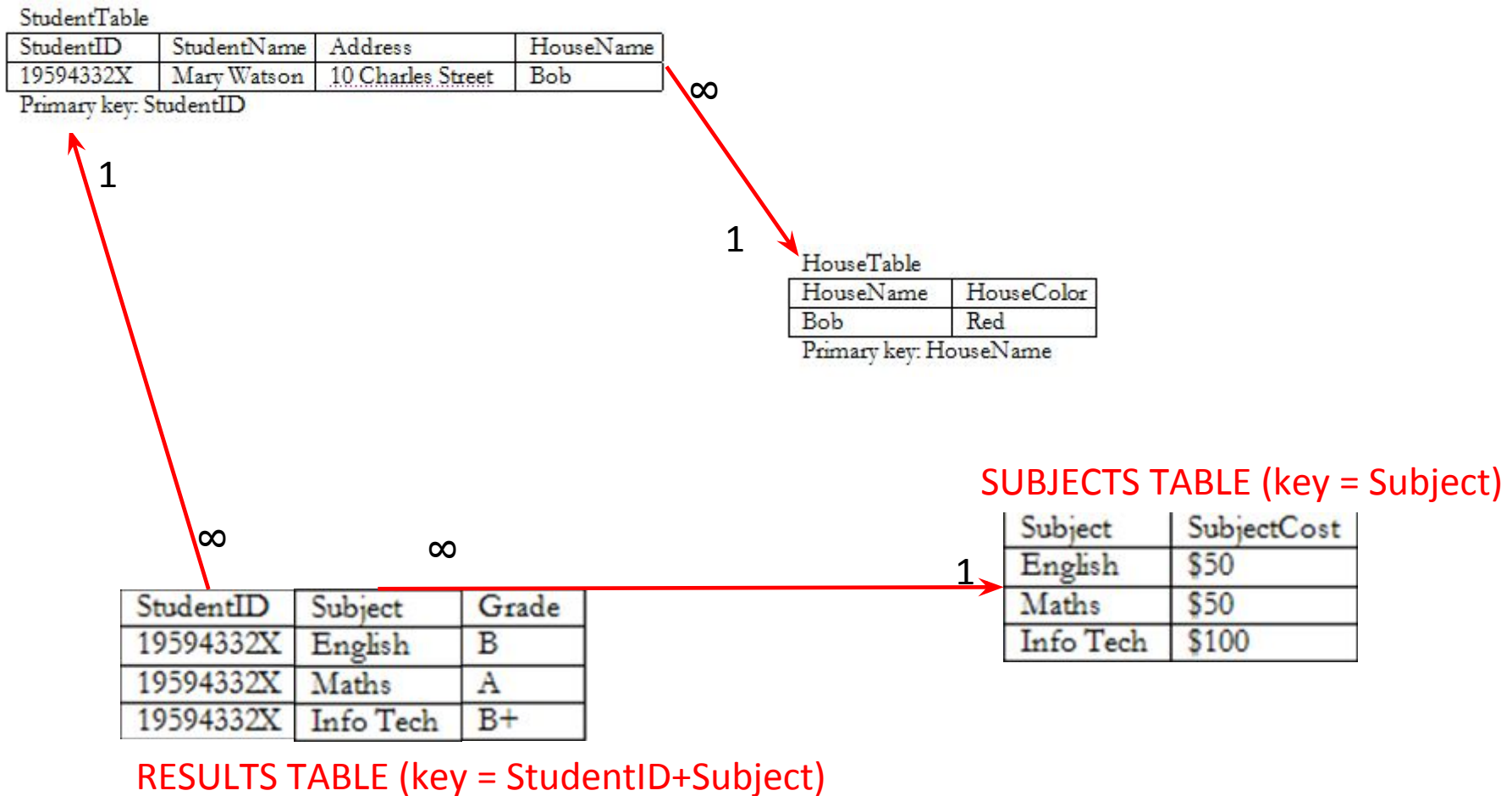
StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

SUBJECTS TABLE (key = Subject)

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

RESULTS TABLE (key = StudentID+Subject)

A 3NF fix



The Reveal=3nf done!

Before...

StudentID	StudentName	Address	HouseName	HouseColor	Subject	SubjectCost	Grade
19594332X	Mary Watson	10 Charles Street	Bob	Red	English	\$50	B
					Maths	\$50	A
					Info Tech	\$100	B+

After...

StudentTable

StudentID	StudentName	Address	HouseName
19594332X	Mary Watson	10 Charles Street	Bob

Primary key: StudentID

1

HouseTable

HouseName	HouseColor
Bob	Red

Primary key: HouseName

∞

1

∞

∞

StudentID	Subject	Grade
19594332X	English	B
19594332X	Maths	A
19594332X	Info Tech	B+

RESULTS TABLE (key =
StudentID+Subject)

1

Subject	SubjectCost
English	\$50
Maths	\$50
Info Tech	\$100

SUBJECTS TABLE (key = Subject)



BCNF

College Enrollment Table

student_id	subject	professor
101	Java	P. Java
101	C++	P. Cpp
102	Java	P. Java2
103	C#	P. Chash
104	Java	P. Java

College Enrollment Table

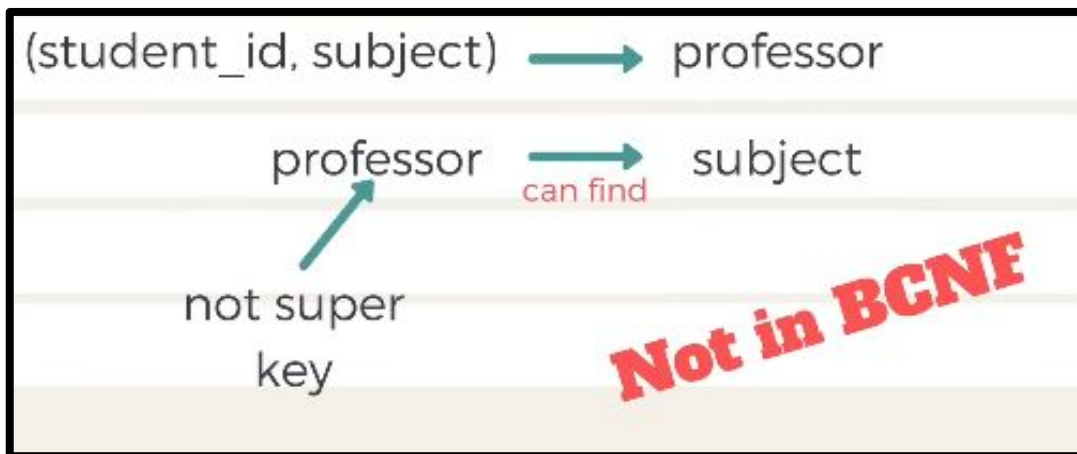
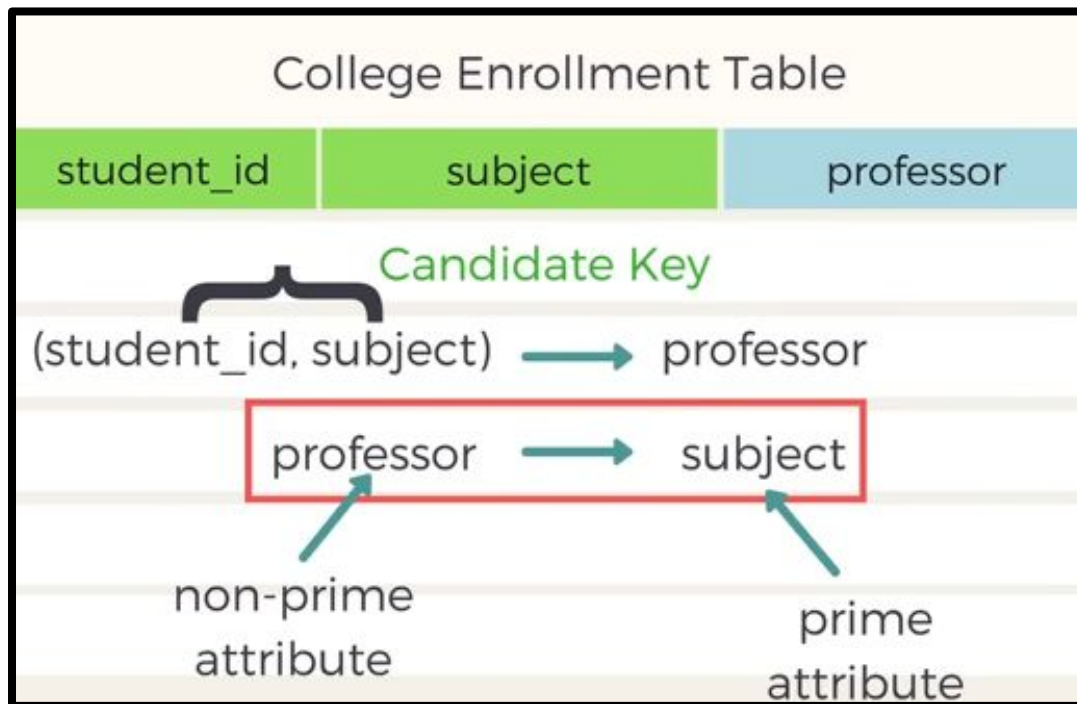
student_id	subject	professor
------------	---------	-----------

(student_id, subject) $\rightarrow$ professor
professor $\rightarrow$ subject

} Dependencies



BCNF





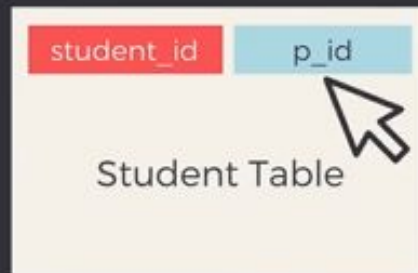
BCNF

How to make the table satisfy BCNF?

College Enrollment Table



Student Table + Professor Table



Now our tables satisfy BCNF



Decomposition

- The only way to avoid the repetition-of-information problem in the *in_dep* schema is to decompose it into two schemas – instructor and *department* schemas.
- Not all decompositions are good. Suppose we decompose

employee(*ID*, *name*, *street*, *city*, *salary*)

into

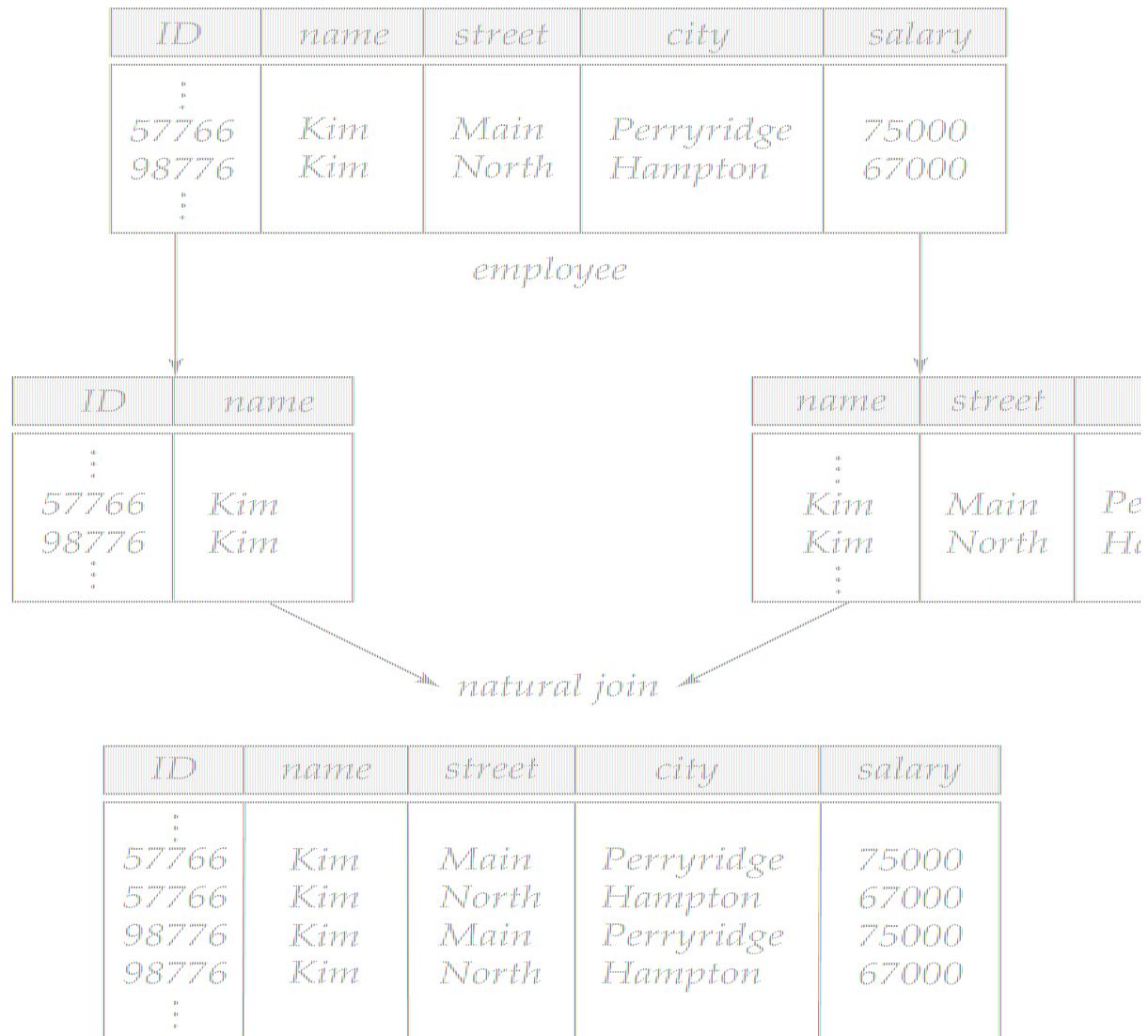
employee1 (*ID*, *name*)

employee2 (*name*, *street*, *city*, *salary*)

The problem arises when we have two employees with the same name

- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

A Lossy Decomposition



Lossless Decomposition

- Let R be a relation schema and let R_1 and R_2 form a decomposition of R . That is $R = R_1 \cup R_2$
- We say that the decomposition is a **lossless decomposition** if there is no loss of information by replacing R with the two relation schemas $R_1 \cup R_2$
- Formally,

$$\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

- And, conversely a decomposition is lossy if

$$r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

Example of Lossless Decomposition

- Decomposition of $R = (A, B, C)$
 $R_1 = (A, B)$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B

Lossless Decomposition

- We can use functional dependencies to show when certain decomposition are lossless.
- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless decomposition if at least one of the following dependencies is in F^+ :
 - $R_1 \cap R_2 \rightarrow R_1$
 - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies

Example

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$
- $R_1 = (A, B), R_2 = (B, C)$
 - Lossless decomposition:
 $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
- $R_1 = (A, B), R_2 = (A, C)$
 - Lossless decomposition:
 $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
- *Note:*
 - $B \rightarrow BC$
is a shorthand notation for
 - $B \rightarrow \{B, C\}$



- R (A B C)
- 1 1 2
- 2 1 2
- 3 2 1
- 4 3 2
- Decompose it R1(A,B) and R1(B, C) Lossless?
- Decompose if R1(A,B) and R1 (A , C) Lossless?

- Short trick -> Just check common attribute is Candidate key or not. If yes then it is lossless.



Dependency Preservation

- Testing functional dependency constraints each time the database is updated can be costly
- It is useful to design the database in a way that constraints can be tested efficiently.
- If testing a functional dependency can be done by considering just one relation, then the cost of testing this constraint is low
- When decomposing a relation it is possible that it is no longer possible to do the testing without having to perform a Cartesian Product.
- A decomposition that makes it computationally hard to enforce functional dependency is said to be NOT **dependency preserving**.



Dependency Preservation , How to Check?

Set of original dependency should be preserved after decomposition of relation

- Let, R (ABCD)
- FD, F: A→B, B→C , C→D, D→B
- R decomposed into R1 (AB) , R2(BC), R3(BD)
- And R1 validates F1, R2 validates F2, R3 validates F3.
 - This decomposition is **dependency preserving**, if
$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$



Dependency Preservation , How to Check?

- Let, $R(ABCD)$
- FD, $F: A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B$
- R decomposed into $R_1(AB), R_2(BC), R_3(BD)$
- For R_1 ,
- Possible dependencies :
- $A \rightarrow B$ But we can't just assume, we need to check their validity using the original FD of R
- $B \rightarrow A$



Dependency Preservation , How to Check?

- For R1,
- Possible dependencies :
- $A \rightarrow B$
- $B \rightarrow A$

But we can't just assume, we need to check their validity using the original FD of R, remember if not valid then dependency will not be preserved!!!!

- So, is $A \rightarrow B$ valid?
- $A \rightarrow B$ exists in F , so $A \rightarrow B$ valid
- Is $B \rightarrow A$ valid?
- Find closure of B using F : $B^+ = BCD$, so $B \rightarrow A$ is not valid
- So, F1: $A \rightarrow B$

F: $A \rightarrow B$
 $B \rightarrow C$,
 $C \rightarrow D$
 $D \rightarrow B$



Dependency Preservation , How to Check?

- For $R_2(BC)$,
- Possible dependencies :
- $B \rightarrow C$, $C \rightarrow B$

But we can't just assume, we need to check their validity using the original FD of R, remember if not valid then dependency will not be preserved!!!!

- So, is $B \rightarrow C$ valid?
- $B \rightarrow C$ exists in F , so $B \rightarrow C$ is valid.
- Is $C \rightarrow B$ valid?
- Find closure of C using F : $C^+ = CDB$, so $C \rightarrow B$ is valid
- So, F2: $B \rightarrow C$, $C \rightarrow B$

F: $A \rightarrow B$
 $B \rightarrow C$,
 $C \rightarrow D$
 $D \rightarrow B$



Dependency Preservation , How to Check?

- For $R_3(BD)$,
- Possible dependencies :
- $B \rightarrow D$, $D \rightarrow B$

But we can't just assume, we need to check their validity using the original FD of R, remember if not valid then dependency will not be preserved!!!!

- So, is $B \rightarrow D$ valid?
- $B^+ = BCD$ using F, So, $B \rightarrow D$ valid
- Is $D \rightarrow B$ valid?
- $D \rightarrow B$ exists in F , so $D \rightarrow B$ valid
- So, F_3 : $B \rightarrow D$, $D \rightarrow B$

F: $A \rightarrow B$
 $B \rightarrow C$,
 $C \rightarrow D$
 $D \rightarrow B$



Dependency Preservation , How to Check?

F: A→B
B→C,
C→D
D→B

F1: A→B

F2:
B→C,
C→B

F3: B→D
D→B

F1 ∪ F2 ∪ F3 :
A→B
B→C,
C→B
B→D
D→B

Remember?

decomposition is **dependency preserving**,
if

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

How to check ?

- check if we can F1UF2UF3 if we take closure of F+.
- So this time, we will check F using F1UF2UF3.



Dependency Preservation , How to Check?

F: A->B
B->C,
C->D
D->B

Can we get A->B from F1UF2UF3?

A+ : ABCD using F1UF2UF3

So , A->B ✓

Can we get B->C from F1UF2UF3?

B+ : BCD using F1UF2UF3

So, B->C ✓

Can we get C->D from F1UF2UF3?

C+ : CBD using F1UF2UF3 , So C->D ✓

Can we get D->B from F1UF2UF3?

D+ : DB using F1UF2UF3 , So D->B ✓

That means, F preserved by F1 U F2 U F3

Remember?

decomposition is dependency preserving,
if

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

How to check ?

- check if we can F1UF2UF3 if we take closure of F+.
- So this time, we will check F using F1UF2UF3.

F1 U F2 U F3 :

A->B

B->C,

C->B

B->D

D->B



Dependency Preservation , How to Check?

- Another example.
- $R(ABCD)$ decomposed into $R_1(AD)$ and $R_2(BCD)$
- Given FDs of R : $AB \rightarrow CD$, $D \rightarrow A$
- Check if dependency is preserved or not?



Dependency Preservation , How to Check?

- Another example.

R(A,B,C,D,E)

F = { A → BCD

B → AE

BC → AED

D → E

C → DE }

decompose it into R1(A,B)

R2(B,C)

R3(C,D,E)



Dependency Preservation , How to Check?

- Another example.
- **$R(A,B,C,D)$**
 $F = \{A \rightarrow B, C \rightarrow D\}$
decomposition **$R1(AB)$ and $R2(CD)$**
- **$R(A,B,C,D)$**
 $F = \{A \rightarrow B, A \rightarrow C, C \rightarrow D\}$
 $R1(A,B,C)$ and $R2(C,D)$



- A relation R (A,B,C,D,E)
- F { A->B, B->C, C->D, D->A}

R is decomposed into R1(A,B,C) and R2(C,D,E). Is the dependency preserving?

R1 (A,B C)

A+ = ABCD A->BC

B+ = BCDA B->CA

C+ = CDAB C->AB

AB+ = ABCD

BC+ = BCDA

CA+ = CABD

R2 (C,D,E)

C+ = CDAB C->D

D+ = DABC D->C

E+ = E

CD+ = CDAB

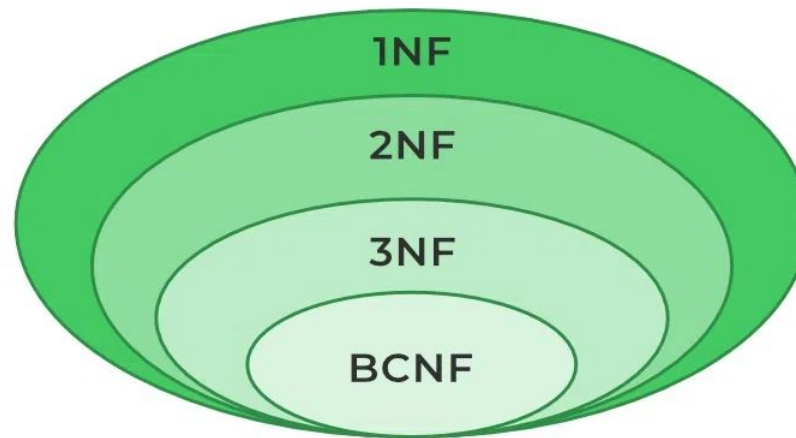
DE+ = DEABC

EC+ = ECDAB

{ A-> BC, B->CA, C->AB, C->D, D->C} PD? YES



Check in Which Normal Form?





Check in Which Normal Form?

- Step-1: Find all Candidate keys in the Relation
- Step-2: Find all prime and non prime attr.
- Step-3: Check from BCNF .
Check which FDs are in BCNF. If any FD is in BCNF, it will be in 2NF and 3NF also. If any one of the FDs can't satisfy the BCNF condition then, the relation is not in BCNF.
- Step-4: if not in BCNF, check for 3NF. FDs satisfying 3nf will also satisfy 2nf.
- Step-5: if not in 3NF , check for 2NF.



Check in Which Normal Form?

- $R(ABCDEF)$
- FD: $AB \rightarrow C$
 $C \rightarrow D$
 $C \rightarrow E$
 $E \rightarrow F$
 $F \rightarrow A$

Check in which normal form?

- Step-1: Find all Candidate keys in the Relation
- $AB^+ = ABCDEF$
- $FB^+ = FBACDE$
- $EB^+ = EBFACD$
- $CB^+ = CBEFAD$
- So, CK (AB, FB, EB, CB)
-



Check in Which Normal Form?

- R(ABCDEF)
- FD: AB $\rightarrow$ C
C $\rightarrow$ D
C $\rightarrow$ E
E $\rightarrow$ F
F $\rightarrow$ A

Check in which normal form?

- Step-2: Find all prime and non prime attr.
- CK (AB, FB, EB, CB)
- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D



Check in Which Normal Form?

- Step-3: Check from BCNF .
Check which FDs are in BCNF. If any FD is in BCNF, it will be in 2NF and 3NF also. If any one of the FDs can't satisfy the BCNF condition then, the relation is not in BCNF.
- CK (AB, FB, EB, CB)
- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D
- FD: AB $\rightarrow$ C ✓
 - C $\rightarrow$ D ✗
 - C $\rightarrow$ E ✗
 - E $\rightarrow$ F ✗
 - F $\rightarrow$ A ✗

BCNF Condition:
L.H.S must be super/candidate key.

Not in BCNF



Check in Which Normal Form?

- Step-4: Check for 3NF.
- CK (AB, FB, EB, CB)
- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D
- FD: AB $\rightarrow$ C ✓ (no need to check again)
 - C $\rightarrow$ D ✗
 - C $\rightarrow$ E ✓
 - E $\rightarrow$ F ✓
 - F $\rightarrow$ A ✓

3NF Condition:

L.H.S must be super/candidate key.

Or,

R.H.S is a prime attr.

Not in 3NF



Check in Which Normal Form?

- Step-5: Check for 2NF.
- CK (AB, FB, EB, CB)
- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D
- FD: AB $\rightarrow$ C ✓ (no need to check again)
 - C $\rightarrow$ D ✗ partial dependency
 - C $\rightarrow$ E ✓ (no need to check again)
 - E $\rightarrow$ F ✓ (no need to check again)
 - F $\rightarrow$ A ✓ (no need to check again)

Not in 2NF

How can we convert it into 2NF?

2NF Condition:

No partial dependency.

Non prime attr. Cannot depend on part of the candidate key.

If R.H.S is non prime then L.H.S must be a candidate key.



Conversion

- Step-5: Check for 2NF.
- CK (AB, FB, EB, CB)
- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D
- FD: AB $\rightarrow$ C ✓ (no need to check again)
 - C $\rightarrow$ D ✗ partial dependency
 - C $\rightarrow$ E ✓ (no need to check again)
 - E $\rightarrow$ F ✓ (no need to check again)
 - F $\rightarrow$ A ✓ (no need to check again)

Not in 2NF

How can we convert it into 2NF? Decompose!!

2NF Condition:

No partial dependency.

Non prime attr. Cannot depend on part of the candidate key.

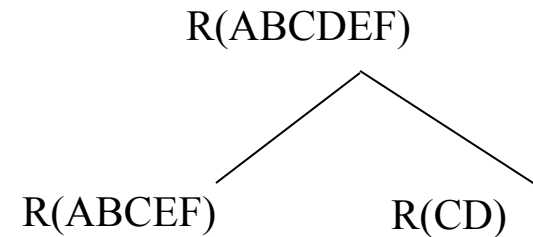
If R.H.S is non prime then L.H.S must be a candidate key.



Conversion

CK (AB, FB, EB, CB)

- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D
- FD: AB $\rightarrow$ C ✓ (no need to check again)
 - C $\rightarrow$ D ✗ **partial dependency**
 - C $\rightarrow$ E ✓ (no need to check again)
 - E $\rightarrow$ F ✓ (no need to check again)
 - F $\rightarrow$ A ✓ (no need to check again)



Not in 2NF

How can we convert it into 2NF? Decompose!!

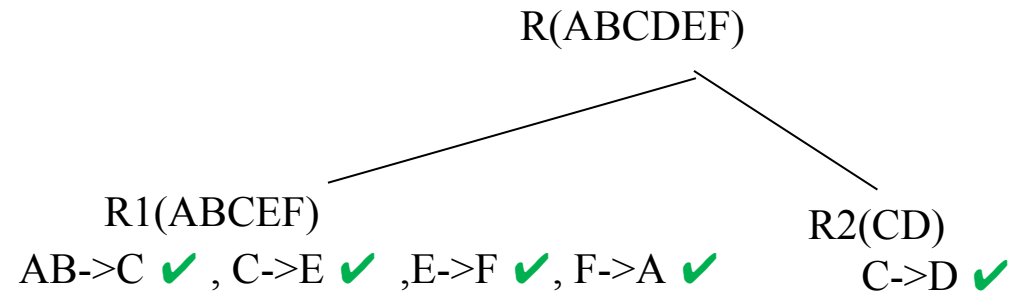
C $\rightarrow$ D is violating the condition.



Conversion

CK (AB, FB, EB, CB)

- Prime Attr. = A, B, C, E, F
- Non Prime Attr = D
- FD: AB $\rightarrow$ C
C $\rightarrow$ D
C $\rightarrow$ E
E $\rightarrow$ F
F $\rightarrow$ A





Conversion

To convert in BCNF.

R1: $AB \rightarrow C$ ✓

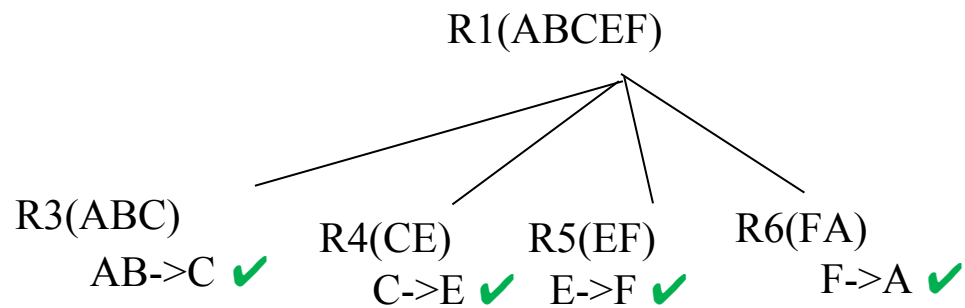
$C \rightarrow E$ ✗

$E \rightarrow F$ ✗

$F \rightarrow A$ ✗

R2: $C \rightarrow D$ ✓

So , R1 and R2 is in 3NF.





Previous Questions

-
- a) Consider the following relation.

$R1(A, B, C, D, E, F)$

FD: {

$AB \rightarrow CD$

$BC \rightarrow DE$

$EF \rightarrow A$

$D \rightarrow E$

$E \rightarrow F$

}

- i) Find out all candidate keys for the given relation.
- ii) Check whether $BE \rightarrow CD$, $AB \rightarrow F$ and $DE \rightarrow AC$ is a valid functional dependency for the relation or not. Justify your answer using Armstrong's Axiom's.
- iii) Find out the canonical cover for the given functional dependencies.
- iv) Check and justify in which normal form the relation is.



Previous Questions

- b) Consider the following relation,

$R2(A, B, C, D, E, F)$

FD: {

$AB \rightarrow C$

$CD \rightarrow E$

$EF \rightarrow AB$

}

Suppose, we want to decompose the relation in following way,

$R21(E, F, A, B)$

$R22(C, D, E)$

Will it preserve the dependencies? Justify your answer with proper explanation.

- c) Check whether the following decomposition of the relation given below is a valid decomposition or not. Justify your answer with proper explanation of lossless decomposition.

$R(A, B, C, D, E, F, G)$

FD: {

$AB \rightarrow C$

$AC \rightarrow DE$

$B \rightarrow F$

$E \rightarrow G$

}

Decompose to: $R1(A, B, C)$

$R2(A, C, D, E)$

$R3(E, F, G)$



Previous Questions

- a) Consider the following relation and find out **4 valid Functional Dependencies** from the given relation:

Publisher			
ID	Name	Rating	Number of Books Published
1	Samin	4.5	30
2	Tuhin	4.9	20
3	Niloy	4.2	12
4	Samin	4.5	5
5	Shakil	4.6	20
6	Niloy	4.1	13



Previous Questions

- b) Consider the following *relation, R* and a set of *functional dependencies*:
- $R = \{A, B, C, D, E, I\}$
 $FD = \{ A \rightarrow C, AB \rightarrow C, C \rightarrow DI, CD \rightarrow I, EC \rightarrow AB, EI \rightarrow C \}$
- Find out all the *Candidate keys* from the given dependencies.
 - Check and justify in which *Normal form* the relation is.
 - Find the *Minimal Cover* of the relation, R
- c)
- Distinguish between *Partial Dependency* and *Transitive Dependency*.
 - Check whether the following *decomposition* of the relation, R will *Preserve the dependencies* or *not*.
- $R = \{A, B, C, D, E, G\}$
 $FD = \{ AB \rightarrow C,$
 $AC \rightarrow B,$
 $AD \rightarrow E,$
 $B \rightarrow D,$
 $BC \rightarrow A,$
 $E \rightarrow G\}$
- Decompose to: R1 (A, B)
 R2 (B, C)
 R3 (A, B, D, E)



Previous Questions

- b) Write short description of the following topics
 - i) Transitive Dependency
 - ii) Insertion anomaly
- c) Consider the relation $R1(A, B, C, D, E, F, G)$ with the following dependency:
 - $AB \twoheadrightarrow CD$
 - $BCD \twoheadrightarrow E$
 - $DG \twoheadrightarrow F$
 - $EF \twoheadrightarrow A$
 - $G \twoheadrightarrow B$Find out all the candidate keys.



Previous Questions

a) Read the following scenario and answer the following question:

You and your teammates, Lionel Messi and Cristiano Ronaldo, are working on a DBMS project for your lab class. The project involves designing a database schema to manage player statistics for Wakanda Premier League (WPL). As part of the project, you need to determine the best decomposition strategy for a given relation R.

Messi and Ronaldo have each proposed a solution for decomposing the relation $R=(A,B,C,D,E)$ with the following functional dependencies:

$A \rightarrow BC$ $CD \rightarrow E$ $B \rightarrow D$ $E \rightarrow A$

Messi's Proposal	Ronaldo's Proposal
$R_1 = (A,B,C)$	$R_1 = (A,B,C)$
$R_2 = (A,D,E)$	$R_2 = (C,D,E)$

Both are convinced their plan is the ticket to victory! As the team captain, it's up to you to settle the score and lead the project to glory. You must carefully evaluate both proposals and find their validity considering factors such as **normalization, data loss, and preservation of functional dependencies**.

- If both solutions are valid then choose the decomposition strategy that is the most suitable for the project and justify your answer.
- If none of them are valid then propose your solution which will overcome the issues of Messi and Ronaldo's proposal.
- Prepare a comparison table which will compare the considering factors of each proposal.

Your decision will determine the direction of the project and contribute to its success.



Previous Questions

Consider the relation $R1(A, B, C, D, E, F, G)$ with the following dependency:

$A \rightarrow B$	$AG \rightarrow C$
$B \rightarrow C$	$AC \rightarrow D$
$CD \rightarrow E$	$F \rightarrow G$
$E \rightarrow F$	$AE \rightarrow B$

- Find minimal cover of the given relation.
- Find the set of non prime attributes of R .



Previous Questions

- a) Suppose we have a relation named R, having attributes as A, B, C, D, E and F. The relation R is having the following functional dependencies,

$A \rightarrow C$

$B \rightarrow DF$

$AB \rightarrow E$

$D \rightarrow F$

Considering the relation with functional dependencies, find if **$CD \rightarrow E$** and **$AB \rightarrow CE$** is valid or not using Armstrong's axioms. [mention the axiom's rule applied in each state]



Previous Questions

b) Consider the functional dependencies of the relation,

R(P, Q, R, S, T, U)

FD: {

P → SQR

Q → T

R → U

PR → U

S → Q

}

- i) Find out all the candidate keys.
- ii) Find out the minimal cover.
- iii) Explain in which normal form the relation is with proper justification.



Previous Questions

- c) Consider a relation **R(L, M, N, O, P, R)** with the following functional dependencies,

L → NR
M → OP
LM → P

If we decompose the relation in **R1(L, N, R, M)** and **R2(M, O, P, L)**, state the validity of this decomposition in terms of lossless and dependency preserving concepts.